

A Unified Framework for Interactive Visual Graph Matching via Attribute-Structure Synchronization

Yuhua Liu^a, Haoxuan Wang^a, Jiajia Kou^a, Ling Sun^a, Heyu Wang^{a,*}, Yongheng Wang^b, Yigang Wang^a, Jinchang Li^c and Zhiguang Zhou^{a,*}

^a Hangzhou Dianzi University, Hangzhou 310018, China

^b Zhejiang Lab, Hangzhou 311121, China

^c School of Data Sciences, Zhejiang University of Finance & Economics, Hangzhou 310018, China

ARTICLE INFO

Keywords:

Machine Learning

Human-Computer

Interaction

Dimensionality Reduction

Graph/Network Data.

ABSTRACT

In traditional graph retrieval tools, graph matching is commonly used to retrieve desired graphs from extensive graph datasets according to their structural similarities. However, in real applications, graph nodes have numerous attributes which also contain valuable information for evaluating similarities between graphs. Thus, to achieve superior graph matching results, it is crucial for graph retrieval tools to make full use of the attribute information in addition to structural information. We propose a novel framework for interactive visual graph matching. In the proposed framework, an attribute-structure synchronization method is developed for representing structural and attribute features in a unified embedding space based on Canonical Correlation Analysis (CCA). To support fast and interactive matching, our method provides users with intuitive visual query interfaces for traversing, filtering and searching for the target graph in the embedding space conveniently. With the designed interfaces, the users can also specify a new target graph with desired structural and semantic features. Besides, evaluation views are designed for easy validation and interpretation of the matching results. Case studies and quantitative comparisons on real-world datasets have demonstrated the superiorities of our proposed framework in graph matching and large graph exploration.

© 2025 Elsevier B.V. All rights reserved.

1. Introduction

The advancement of graph data management techniques has led to the widespread collection of numerous graph datasets, catering to the data-dependent investigations and applications across various domains, such as social connections [1], business relations [2], and knowledge derivations [3]. Graph matching frequently serves as an exploratory method for retrieving desired complete graphs from a large-scale graph dataset with extensive graphs based on various similarity measures.

Graph matching prioritizes structures, with their similarity measured in terms of topological features or learned vectors. For instance, in a chemical compound database D , biochemists

often retrieve and match the desired compounds in D based on the structure similarity. To achieve a more comprehensive representation of graph structures in matching tasks, techniques such as graph kernel [4] and graph representation learning [5] have been applied alongside traditional graph similarity metrics, including maximum common subgraph (MCS) [6] and minimum graph edit distance (MGED) [7].

However, in practical scenarios, graph nodes are depicted with rich attributes, which are also important for graph matching [8]. For instance, in the analysis of protein-protein interaction networks, incorporating protein attributes enables biologists to achieve more precise identification of protein complexes [9]. Similarly, within a large-scale air propagation network, the

air transmission structure and the proportion of urban air composition constitute important propagation patterns [10].

Evidently, the integration of both structural and attribute information into a unified model provides substantial advantages for reliable graph matching. Nevertheless, it is challenged from three aspects as follows:

C1: Structures and attributes exhibit distinct and heterogeneous features. Constructing an attribute-structure synchronization considering the correlation between them toward yielding better-quality matching results is challenging.

C2: Graph query language serves as a prevalent means of retrieving graphs. However, it is difficult to learn and use for desired graph matching, especially for those non-experts. Providing visual graph querying interfaces is a feasible manner to synchronize structures and attributes, and empower non-experts to construct queries.

C3: Graph matching, employing attribute-structure synchronization, often retrieves numerous candidate graphs. It becomes essential to furnish users with quantitative and visually accessible recommendations, aiding them in selecting the most suitable graph to meet their specific requirements.

To tackle the above challenges, we propose a method to interactively query graphs considering attribute-structure synchronization. For a specific graph database, our method will train a model to learn the potential correlation between attribute and structure. There are two stages: 1) the representations of structure features and attributes of graphs and 2) the fusion considering the intrinsic correlation between two dimensional features. Graph representation learning is commonly used to learn the structure feature of graphs. In the first stage, Graph2vec is employed to learn the structure feature and the multiple attributes of nodes are statistics as multi-dimensional vectors. Then Canonical Correlation Analysis (CCA) is utilized to integrate the acquired structural information and multiple attributes into a synthetic model. This model is then subjected to joint dimensionality reduction, resulting in the projection of graphs into a low-dimensional feature space, where those graphs with shared structural and attribute characteristics are positioned close to each other (**C1**). After obtaining the joint representation of the graph database, our method facilitates the measurement of distance-based similarity, enabling efficient graph matching.

To further facilitate graph matching with attribute-structure synchronization, we have devised two visual matching schemes. The one scheme provides multiple projection views to assist users in quickly traversing, filtering and searching for the target graph during the matching process. The other provides a variety of customizable panels, empowering users to define a new target graph with specific semantic and structural features, and then retrieve those graphs matching the manually-specified target (**C2**). Multiple coordinated views and a series of interactions have been designed to support the visual evaluation and exploration of matched results. Users can readily assess the similarities between the target graph and the matched results using node-link diagram and parallel coordinates (**C3**). Finally, a visual analysis system is developed to integrate the above matching model, visualization views and interactions, enabling users to accurately match and explore large-scale multivariate

graph data, as shown in Figure 3. The primary contributions of this paper are summarized as follows:

- An attribute-structure synchronization based on Canonical Correlation Analysis is proposed for credible graph matching.
- Visual interfaces and user-friendly interactions are designed to replace conventional graph query languages for graph matching, evaluation, and exploration.
- Case studies and quantitative comparisons based on real-world datasets are conducted to verify the effectiveness and convenience of our system.

2. Related work

2.1. Graph Matching

Graph matching can be divided into two categories, such as decision version and matching version. The decision version is to determine whether the query graph exists in a graph database. The matching version refers to discovering the embedding of the query graph in a large graph. Previous matching methods are categorized as filter-then-verify (FTV) [11] and direct subgraph isomorphism (SI) [12]. The FTV methods include two stages: filtering and verification. In the filtering stage, the query graph is decomposed into features such as paths [13], trees [14], or their combinations. The graphs without these features are filtered out, and the remaining graphs are retrieved to form a candidate set. In the verification stage, the final querying results are generated by conducting a subgraph isomorphism test to the query graph against the candidate set. However, these technologies suffer from limited filtering capabilities and high cost of verification [15]. To address this issue, the direct SI method [16], was proposed, which accelerates the SI test by locating the best candidate nodes for each graph, and joins these nodes to match the query [17]. The proposed FTV and direct SI are both applied to address the two versions of graph matching problems.

In the field of visualization, graph matching belongs to the matching version, i.e., exploring the embedding of a target structure/subgraph from a large network [18, 19]. For example, VISAGE [20] performs graph querying on a large graph with nodes of different types and attributes. g-Miner [21] supports mining groups from a multivariate network with the capability to adapt to complex or dynamic requirements of the desired group. Wong et al. [22] developed a visual analytics paradigm, which cannot only rapidly preprocess a graph with billions of edges but also support graph query and interactive visualization on the processed graph. Chen et al. [23] developed a visual exploration system that suggests appropriate structures upon user-specified exemplars in a large network. VIGOR [24] is an exemplar-based interaction technique to support top-down and bottom-up searches for paradigm patterns from a large network. Song et al. [25] use graph representation learning to match graphs with similar subgraph structures. However, most existing works treat structure and attribute either independently or fuse them through predefined or tightly coupled models, limiting generalizability and interpretability. Our approach introduces a modular and extensible framework that decouples structure embedding and attribute alignment via

Canonical Correlation Analysis (CCA), enabling flexible integration of heterogeneous graph data. It uniquely combines embedding, synchronization, and visual analytics to support interpretable and interactive graph-level matching.

2.2. Graph Representation Learning

Graph representation learning (GRL) focuses on how to embed graph structures into a vector space while preserving the structural property, which can be roughly divided into three categories [26, 27]. (1) Factorization based GRL methods represent the connection between nodes in the form of an adjacency matrix, Laplace matrix [28], etc., and then factorize these matrices to obtain node vectors. (2) Random-walk based GRL methods generate node sequences by a random walk on the graph and obtain the representation results so that nodes have similar embeddings if they tend to co-occur on random walks, such as DeepWalk and node2vec [29]. Inspired by the random walk strategy, Graph2vec is proposed to learn representations of whole graphs by characterizing each graph as a sequence of rooted subgraphs. (3) Deep-learning based GRL methods utilize the ability of deep auto-encoder to generate vector representations, which can capture the nonlinear structure information in graphs [30].

Attribute graph embedding has attracted much attention in recent years, since it's strongly believed that graph semantics (e.g., node attributes) carries complementary knowledge beyond the topology of graphs. Two categories of methods are proposed. (1) The matrix-decomposition based methods [31] are to add the attribute information of nodes to the process of matrix decomposition, for further generating new node vectors. (2) The deep-learning based method [32] projects the attribute information and topology information of nodes into the same semantic space. However, the above methods focus on the embedding of nodes in attribute graphs, which is different from our study which applies global attribute graph embeddings to graph matching.

2.3. Graph Similarity Measure

It is significant for graph matching to define a feasible similarity metric [33] which is measured with the difference between two graphs by means of quantitative similarity values. GQL is a graph query language, which cannot only support filtering nodes with specific attribute values, but also defines semantic rules for graph matching, such as Cypher [34], Gremlin [35] and SPARQL [36]. In addition, some graph similarity studies based on semantic features have been proposed. For example, Khan et al. [37] proposed a subgraph matching cost metric which unifies both structure and node label similarity. Zhao et al. [38] took the local features of nodes as basic graph indexing units.

Structure-based similarity measures are widely investigated in the field of graph databases, which can be divided into two categories depending on whether the space is transformed in the measurement process: direct methods and indirect methods. Direct methods measure the structural similarity through a graph matching procedure. The most widely-used direct methods are maximum common subgraph (MCS) [6] and minimum

graph edit distance (MGED) [7]. By contrast, indirect methods [39] transform graphs into vectors, and then estimate the similarity between graphs in the vector space. Currently, graph kernels and graph representation learning have also been employed for measuring graph similarities. Graph kernel measures the similarity between a pair of graphs based on the recursively decomposed substructures of graphs [40]. Graph representation learning [25] uses the node-level or graph-level vectorized representation of graphs to measure their similarity. With the development of deep learning technology, graph neural network has also been successfully applied to learn the similarity between graphs [41].

3. Requirement analysis and system overview

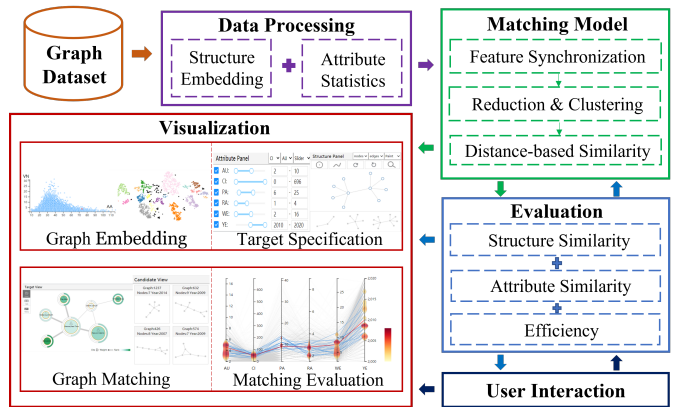


Fig. 1. The pipeline of our graph matching system based on attribute-structure synchronization. Users first import a dataset and configure the matching model via the control panel. A target graph can be selected from the projection view or constructed manually. The selected graph is displayed in the target view for detailed inspection. Once matching is triggered, candidate graphs with similar structure and attributes are retrieved and presented for comparison.

3.1. Requirement Analysis

In this study, we collaborate with three domain experts (E_1 , E_2 and E_3) to identify key challenges in graph matching. E_1 specializes in graph visualization, E_2 is a senior data analyst focusing on social networks, and E_3 is a social science professor experienced in genealogical data. The design process involves two phases of expert interviews. In Phase I, experts highlighted that combining structural and attribute features is essential for graph matching, as both are critical for graph representation and insight discovery. Phase II focused on iteratively developing the system based on expert feedback, particularly for visual matching and result evaluation. From the interviews, we identify four core tasks:

T1: Extracting structure features of graphs. Traditional matching methods rely on substructures like paths or trees but struggle to incorporate attributes. Graph embedding transforms structures into vectors, enabling attribute-structure synchronization. Thus, selecting a suitable embedding method is essential for reliable graph matching.

T2: Attribute-structure synchronization for graph matching. Graph embedding methods transform graphs into

a vector space, while the associated attribute information represents graphs in the attribute space. Quantifying and capturing the correlation between these two spaces is a complex challenge. Simply concatenating the structure feature and the attribute feature cannot be sufficiently helpful to make the fused feature effective for the matching purpose [42]. Therefore, a unified embedding space is needed to preserve both structural and attribute similarities.

T3: Evaluation of graph matching results. Locating graphs that closely resemble the target graph is achieved by quantifying the distance between graphs and identifying the nearest neighbors within the attribute-structure synthetic space. Such distance-based graph matching needs to be further verified. Matching results should be visually interpretable, and similarity metrics are needed to help users assess their validity.

T4: Graph matching framework. Current graph matching relies heavily on complex query languages, limiting usability for non-experts. Visual interfaces help but lack interactivity. A data-driven, user-friendly tool is needed to support exploratory matching based on structural and semantic conditions.

3.2. System overview

Motivated by the identified requirements, we design a visualization framework enabling users to conduct graph matching based on attribute-structure synchronization. The system pipeline is presented in Figure 1. First, a large-scale graph dataset is loaded into the visualization system, and two categories of features are calculated in advance, including structure features represented by the embedding technology, Graph2vec (T1), and attribute features obtained with statistical methods. Then, a novel graph matching model is conducted to combine those two different features into a synthetic embedding space by the CCA-based feature fusion, for achieving distance-based similarity measures after joint dimensionality reduction (T2). Furthermore, a set of well-structured views and interactions are designed for users to match a target graph of interest, evaluate and explore the matching results over this large-scale graph dataset (T3, T4). We also conduct a quantitative experiment with specific metrics on two real-world datasets to assess the quality of the resultant matching results based on our method in terms of structure similarity and attribute similarity (T3).

4. Graph matching model

In this section, we detail our graph matching model based on the attribute-structure synchronization as shown in Figure 2.

4.1. Structure feature extraction

Most graph embedding technologies focus on the node-level structure instead of the global structures of graphs. For this reason, the graph-level embedding technology, Graph2vec, is introduced as our structural feature embedding model. Besides, Graph2vec is a data-driven distributed representation learning approach. Its true superiority of efficiency and robustness has been revealed when training large volumes of graphs. The Graph2vec algorithm mainly consists of two steps as follows:

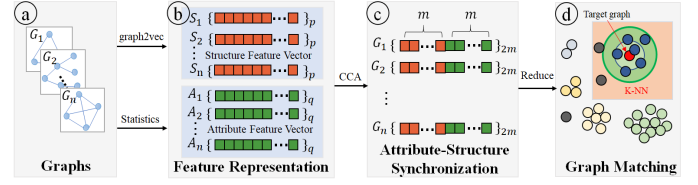


Fig. 2. Given a set of n graphs (a), we extract the topological feature vectors by Graph2vec and the attribute feature vectors by a statistical method (b). Next, the two kinds of vectors with different lengths are translated into new uniform-length vectors and then combined into a synthetic embedding space by CCA (c). A distance-based graph matching scheme is conducted to the space for generating the matching result of a target graph (d).

Step1: Rooted subgraph generation. To facilitate learning graph embedding, a rooted subgraph around each node of one graph need to be generated by such a function, GetWLSubgraph(n, G_i, d), which follows the WeisfeilerLehman relabeling process [43]. Where n is a rooted node, G_i is the graph from which the subgraph is extracted, and d is the degree of the intended subgraph. First, we obtain all neighbors of the rooted node n with the breadth-first traversal. Then we get the degree $d - 1$ subgraph of each neighboring node until $d = 0$. In this recursive process, we concatenate the list which is used in advance to store subgraphs for generating the d -degree subgraph $sg_n^{(d)}$ of n . Given the graph G_i , a set of rooted subgraphs around other nodes in G_i can be generated in the same way, thus the context of G_i is denoted as $c(G_i) = \{sg_1, sg_2, \dots\}$.

Step2: Vectorized representation. To exploit the way the rooted subgraphs compose graphs to learn their embedding, a skip-gram architecture is utilized in the model of Graph2vec to train the distributed representation of subgraphs. The objective of the skip-gram model is to maximize the following log probability:

$$\sum_{j=1}^{l_i} \log P_r(sg_n | g_i) \quad (1)$$

where sg_n is the rooted subgraph of n in G_i generated through the above step, P_r denotes the probability that sg_n appears in g_i , l_i is the total number of rooted subgraphs extracted from graph g_i and j indexes over these subgraphs. The skipgram model captures shared similarities between rooted subgraphs, so that graphs consisting of similar rooted subgraphs are represented close to each other. Finally, we get a N_s -dimensional structure vector $S_i = \{S_{i1}, S_{i2}, \dots, S_{iN_s}\}$, which represents the global structure of graph G_i .

4.2. Attribute-Structure synchronization

In this section, we will introduce how to extract the attribute features from graphs and combine them with the structural features.

1) Attribute feature extraction: The attributes of graphs are extracted based on the domain knowledge of graphs, including micro-level and macro-level. The macro-level attributes refer to global characteristics of graphs, such as the total number of papers published by the authors in a co-authored network, the timespan of a family obtained by subtracting the birth year of the first ancestor from the death year of the last offspring in the

genealogical trees, etc., which reflect the graph-oriented information. In contrast, the micro-level attributes can be obtained by directly summing or averaging the values associated with nodes, such as the average age of users in a social community, and the average working hours of members in a working team.

From the micro-level and macro-level attributes, users can choose appropriate attributes to solve different analysis tasks. For example, to identify teams with strong scientific research ability in a co-author dataset, a variety of attributes can be considered, such as the total number and total cites of papers published by team members, the average author ranking of team members, and the cooperative relationship between team members. When studying the correlation between research productivity and age composition for research teams, the number of published papers and the average age of team members need to be focused on. Finally, these macro-level and micro-level attributes constitute an attribute vector of each graph, $A_i = \{a_{i1}, a_{i2}, \dots, a_{iN_A}\}$, where a_{ij} refers to the value of attribute j , N_A refers to the number of attributes which are extracted from the graph G_i .

2) CCA-based Attribute-Structure Synchronization: Canonical Correlation Analysis (CCA) [42], coupling dimension reduction and feature fusion together, can effectively fuse two different features by maximizing their canonical correlation, which is widely used for image and video retrieval, indexing and annotation. Considering its promising performance in classification and identification, we use CCA to perform joint dimensionality reduction by preserving the correlation between attribute features and structure features.

CCA seeks linear projections of the structural and attribute feature spaces such that the resulting canonical variables are maximally correlated. The latent shared components identified by CCA correspond to underlying factors that simultaneously influence both the graph topology and its attribute distribution. For example, a latent component may capture a functional cluster (e.g., a research topic or organizational unit) that manifests as both a tightly connected subgraph (structure) and a set of recurring attribute patterns (e.g., author affiliation, node roles).

Based on the feature extractions as discussed above, suppose we have obtained the structure feature vectors $\{S_1, S_2, S_3, \dots, S_M\}$ and the attribute feature vectors $\{A_1, A_2, \dots, A_M\}$ of all graphs obtained by the previous two sections in a large graph dataset, where M is the number of graphs. Let S and A be two multidimensional variables in the two sets of feature vectors, and N_S and N_A are their dimensions respectively. Then we normalize different feature value ranges or scales of S and A so that the resulting features have zero mean and unit variance. CCA aims to find two linear transformation matrices, H_S and H_A , respectively for S and A , to maximize the mutual information between the transformed vectors S' and A' .

$$S' = H_S S, A' = H_A A \quad (2)$$

We refer to the pair (S', A') as the CCA transform of S and A . The dimension of the transformation matrix H_S and H_A are $m \times N_S$ and $m \times N_A$ respectively ($m \leq \min(N_S, N_A)$), which are

represented as follows:

$$H_S = \begin{bmatrix} h_{S1}^T \\ h_{S2}^T \\ \vdots \\ h_{Sm}^T \end{bmatrix}, H_A = \begin{bmatrix} h_{A1}^T \\ h_{A2}^T \\ \vdots \\ h_{Am}^T \end{bmatrix} \quad (3)$$

The rows of these two matrices, $\{h_{Si}\}$ and $\{h_{Ai}\}$ ($1 \leq i \leq m$), are considered as CCA basis vectors. The first pair of these basis vectors, $\{h_{A1}\}$ ($1 \leq i \leq m$), are obtained by the following formula :

$$(h_{S1}, h_{A1}) = \arg \max_{(h_S, h_A)} \text{Corr}(h_S^T S, h_A^T A) \quad (4)$$

That means the projections of S and A along the directions represented by (h_{S1}, h_{A1}) are maximally correlated. The first pair of canonical components can be computed by $S'_1 = h_{S1}^T S$, $A'_1 = h_{A1}^T A$. Next, the second pair of CCA basis vectors can be extracted by maximizing the same correlation on the basis of ensuring that they are not correlated to the first pair of canonical components. Thus the remaining canonical pairs can be extracted by the same procedure. The basis vectors are often computed by solving the following equivalent eigenvalue problem relating to the joint covariance matrix C , under the constraints of $h_S^T h_S = 1$ and $h_A^T h_A = 1$.

$$C = \begin{bmatrix} C_{SS} & C_{SA} \\ C_{AS} & C_{AA} \end{bmatrix} \quad (5)$$

where C_{SS} and C_{AA} are the within-set covariance matrices, C_{SA} or C_{AS} is the between-set covariance matrix. Furthermore, we obtain the eigenvectors, namely the normalized CCA basis vectors, and each associated eigenvalue γ_i , $i = 1, 2, \dots, N$, represents the canonical correlation.

Finally, for each graph in the large graph dataset, the original attribute features and structure features can be transformed by CCA to a pair of new vectors with the same length, which is then directly concatenated as an attribute-structure fused vector.

4.3. Graph matching

Based on the attribute-structure synchronization using CCA, all graphs are represented as high-dimensional fused vectors by synthesizing structures and attributes. To support exploratory analysis, we conduct clustering on the fused vectors to help users gain an overview of the embedding space. By observing representative graphs within different clusters, users can develop an initial understanding of how structural and attribute features are distributed across the dataset, which facilitates informed exploration and comparison. The density-based clustering algorithm DBSCAN [44] and the distance-based clustering algorithm K -means [45] are both used here. Thus, users can decide which clustering method to use and adjust their parameters according to the distribution characteristics of graphs in the projection space. When a graph is specified as the target graph, all graphs within the cluster this target graph belongs to are regarded as the matching results. Furthermore, an alternative graph matching is based on the K -nearest neighbor (k -NN), which takes the first K (users can specify the number of matching graphs) graphs that are closest to the target graph as the matching results in the original high-dimensional vector space obtained by CCA.

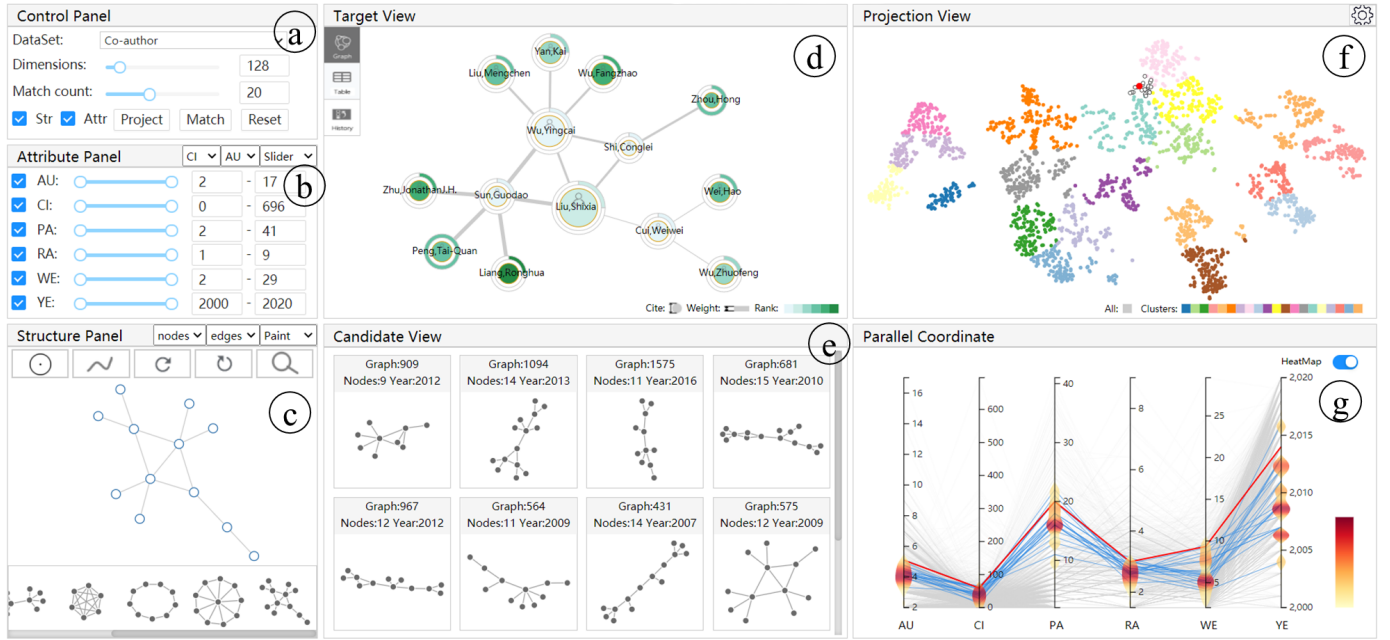


Fig. 3. A case study is conducted to retrieve research communities with similar collaboration and attributes from a large amount of co-author network datasets. (a) The control panel enables users to specify the parameters of our graph matching model. (b) The attribute panel enables users to filter attributes and specify their value ranges. (c) In the structure panel, users can specify desired structures as the target graphs. (d) The matched graph of interest is highlighted in the target view with attributes encoded with different visual elements. (e) The candidate view presents the other matched graphs in the form of a structure overview. (f) Graph2vec is conducted to combine the representations of structures and attributes, and transform graphs into a projection view in which graphs located nearby share similar structures and attributes. (g) The parallel coordinate view is designed to present detailed attributes of the matched graphs.

5. Visual Interfaces

We develop a visual graph matching system that consists of multiple coordinated views (target view, candidate view, projection view, and parallel coordinate view), enabling users to perform graph matching and evaluate the results from both structural and attribute perspectives in an exploratory manner.

5.1. Interactive Graph Matching

Users can specify the target graph through two modes: navigation & clustering-based graph matching, and feature-definition-based graph matching.

5.1.1. Navigation & Clustering-Based Graph Matching

To support users in exploring the entire dataset and identifying graphs of interest, we provide two overview visualizations—the 2D attribute scatter-plot view and the projection view—along with a control panel that allows users to configure matching and visualization parameters.

Control Panel. The control panel (Figure 3(a)) enables users to configure key parameters that influence the graph matching process. It allows switching among three modes: *structure-based*, *attribute-based*, or a *combined structure-attribute* fusion. If both are selected, the system activates a CCA-based model to embed all graphs into a shared latent space.

Attribute Scatter-Plot View. The 2D attribute scatter-plot view presents graphs as data points based on user-specified attributes along the x- and y-axes (Figure 6(a)). This helps users understand the overall attribute distribution and inter-attribute

relationships. Both the attribute and structure panels offer this view: the former visualizes semantic attributes, while the latter shows structural metrics (e.g., number of nodes).

Projection View. The projection view (Figure 3(f)) offers a two-dimensional summary of the dataset using t-SNE [46], where the distance between scatter points reflects graph similarity. Users can toggle among structural, attribute, or fused embeddings as configured in the control panel. The system supports clustering via DBSCAN or *k*-means, and cluster memberships are color-coded to reveal graph groups and outliers.

5.1.2. Feature-Definition-Based Graph Matching

Users may also construct a target graph manually by defining its structure and attribute features. Attributes are adjusted using sliders in the attribute panel (Figure 3(b)). Structural exemplars can be drawn manually or selected from common patterns (templates) extracted from frequent Graph2vec clusters (Figure 3(c)). For these newly created graphs, Graph2vec is applied online to compute their embeddings for matching.

5.2. Graph Matching Evaluation

Target View. When a target graph is selected—from either the projection view or through custom creation—it appears in the target view (Figure 3(d)). Two layout options are provided: a graph layout and a tree layout. The tree layout highlights temporal or hierarchical structures along a time axis. Nodes and edges are encoded with color, size, and shape to reflect

attributes and structure. Legends assist interpretation, and interactions like zooming and panning support in-depth analysis.

Candidate View. The candidate view (Figure 3(e)) shows all matched graphs using node-link diagrams. Candidates are ranked by similarity and the number displayed is adjustable. Selecting a candidate updates the target view for side-by-side comparison.

Parallel Coordinates View. To compare attribute similarity, the parallel coordinates view (Figure 3(g)) plots graphs as polylines across multiple attribute dimensions. A density stream overlays each axis, encoding the distribution of values by color and width. When matches are identified, their lines are highlighted, and the streams update to reflect the attribute alignment with the target.

6. Evaluation

Our visual graph matching system is developed by a classic web-based framework "flask+d3.js+python+MongoDB". Quantitative comparisons of two datasets are conducted to evaluate the effectiveness of our matching method. Case studies and expert feedback are also detailed in this section.

6.1. Datasets

Two real-world attributed graph datasets are applied to conduct case studies.

Genealogy dataset: The China Multigenerational Panel Dataset-Liaoning (CMGPD-LN) [47] is transcribed from the population registers compiled by the Qing Dynasty government in Liaoning Province, northeastern China, from 1749 to 1909. This dataset, with more than 1.5 million records, provides socioeconomic, demographic, and other characteristics for over 260,000 individuals. According to the relationship between individuals, we build approximately 12,000 family trees. In addition, five attributes are extracted and constructed from member information to characterize the features of families, which include the timespan of a whole family (TS), the average age of family members (AA), the number of family members who have been officials (PN), the number of villages where family members have been lived (VN), and the average gap between father and son (AG).

Co-author network dataset: The papers published from 2001 to 2020 in three research areas of visualization (TVCG), data mining (KDD), and human-computer interaction (CHI) are crawled from Web of Science. For each year, we construct a large co-author network according to the co-authorship between scholars of the year. Then the module-based community detection method [48] is applied to all the networks, and about 4,000 small co-author networks, namely research communities, are obtained. We construct multiple attributes for each research community, such as the average number of authors per paper (AU), the average citation count per paper (CI), the total number of published papers (PA), the publication year (YE), the average author rank (RA), and the average number of co-authored papers between a pair of team members (WE).

6.2. Quantitative comparison

We present our experimental studies for the proposed graph matching method based on CCA, which supports attributed graph matching through attribute-structure synchronization. We compare our method with four matching strategies. (1) **Structures (Str)** measures graph similarity by the graph embedding technology, Graph2vec, without considering graph attributes. (2) **Attributes (Attr)** measures graph similarity based on multi-dimensional attributes, without considering graph structures. (3) **Direct concatenating (DC)** directly concatenates the high-dimensional structure vectors (learned from Graph2vec) and low-dimensional attribute vectors, then jointly reduces the dimensions to measure the graph similarity. (4) **Indirect concatenating (IDC)** reduces two different feature vectors into a unified dimension and concatenates them for measuring the graph similarity.

For each dataset, we first randomly select twenty graphs as the target graphs and apply the K -nearest neighbor to find their matching results in the embedding space generated by the above matching methods. To further compare the effectiveness of different graph matching methods, we respectively evaluate the quality of matching results from aspects of structure and attribute. (1) **Structure similarity.** As a classic structure similarity measure, graph editing distance (GED) [49] refers to the minimum operation cost (addition, deletion, and substitution) of transforming from one graph to another. We compute the average GED between each target graph and its matched graphs as the evaluation metric in terms of structure similarity. (2) **Attribute similarity.** Euclidean distance is a widely used method to measure the similarity between multi-dimensional vectors [50]. The average Euclidean distance of the attribute vectors between each target graph and its matched graphs is considered as the evaluation metric in terms of attribute similarity.

Table 1 summarizes the experimental results on different datasets under various K values (K -nearest neighbor). For the larger genealogy dataset, higher K values were used. Results show that our method outperforms Str, Attr, DC, and IDC in structure similarity overall. When $K = 20$, it slightly lags behind Str and DC, but with increasing K , it consistently outperforms all methods. In attribute similarity, our method performs better than Str, DC, and IDC, though slightly worse than Attr, indicating its balanced consideration of both structure and attributes. Notably, it consistently surpasses DC and IDC across both metrics, highlighting its improvement over conventional fusion methods. Similar trends are observed on the coauthor network dataset, where our method even surpasses Attr in attribute similarity, suggesting CCA's effectiveness in maximizing mutual information between structure and attributes. We also evaluated Kernel CCA (KCCA), but it performed poorly (details in supplementary material), likely due to its high-dimensional mapping introducing redundancy and computational overhead, while disrupting the linear correlation between structure and attributes. These results confirm that our CCA-based matching effectively balances and preserves structural and attribute similarities.

To assess the robustness of our CCA-based framework under different neighborhood sizes, we conducted experiments on

Dataset	k	Type	Str	Attr	Our	DC	IDC
Genealogy Dataset	20	Str-Sim	9.64	10.59	10.01	9.74	40.06
		Attr-Sim	63.08	7.02	12.53	20.89	21.66
	40	Str-Sim	9.29	9.43	8.95	9.01	9.11
		Attr-Sim	61.83	7.97	13.04	20.97	22.15
	60	Str-Sim	7.97	7.86	7.45	7.79	7.92
		Attr-Sim	63.69	10.20	15.36	22.49	24.17
Co-author Network Dataset	5	Str-Sim	5.81	6.04	5.92	5.90	5.99
		Attr-Sim	35.61	13.52	9.43	15.39	20.47
	10	Str-Sim	6.16	6.17	6.14	6.12	6.24
		Attr-Sim	36.19	17.85	11.60	19.90	23.15
	15	Str-Sim	5.99	5.92	5.90	5.97	6.01
		Attr-Sim	38.39	20.38	12.91	22.05	25.57

Table 1. Quantitative Comparison

the Co-author dataset with $k \in 5, 10, 15$. As shown in Table 2, our method consistently achieves low structure and attribute matching errors, outperforming KCCA, DC, and IDC. These results confirm the framework’s effectiveness across varying local graph contexts and support its generalizability.

k	Type	Str	Attr	Our	KCCA	DC	IDC
5	Str	5.81	6.04	5.92	6.79	5.90	5.99
	Attr	35.61	13.52	9.43	20.72	15.39	20.47
10	Str	6.16	6.17	6.14	6.10	6.12	6.24
	Attr	36.19	17.85	11.60	22.16	19.90	23.15
15	Str	5.99	5.92	5.90	5.93	5.97	6.01
	Attr	38.39	20.38	12.91	22.91	22.05	25.57

Table 2. Robustness Evaluation of CCA

6.3. Case studies

Given the above two real-world attribute graph datasets, case studies are conducted to demonstrate the effectiveness of our attribute-structure synchronization graph matching system.

1) Genealogy matching: We invited E_3 , a social science expert, to explore the genealogy dataset (D1) using our system. He loaded the dataset via the control panel and examined the family distribution in the projection view (Figure 4(a)). By clicking several points, he noticed the structural complexity of family trees decreased from left to right. He commented, “The distribution of structures shows that the attribute-structure synthetical space preserves the structure similarity well on the whole.”

Next, E_3 explored a cluster and found that nearby points (e.g., T1 and T2 in Figure 4(c, d)) shared similar structures and attributes. The target view detailed the structure and attributes of nodes: node size and shape encoded lifespan and gender (circle for female, square for male), color indicated village, edge width represented the number of offspring, and officials were marked. E_3 noted, “Structure is abstract to me, but the node-link graph makes it intuitive. Attribute mappings also reveal deeper family characteristics.” Later, he identified a third family tree, T3 (Figure 4(e)), structurally similar to T1 and T2. Using the parallel coordinate view (Figure 4(b)), he compared their attributes and

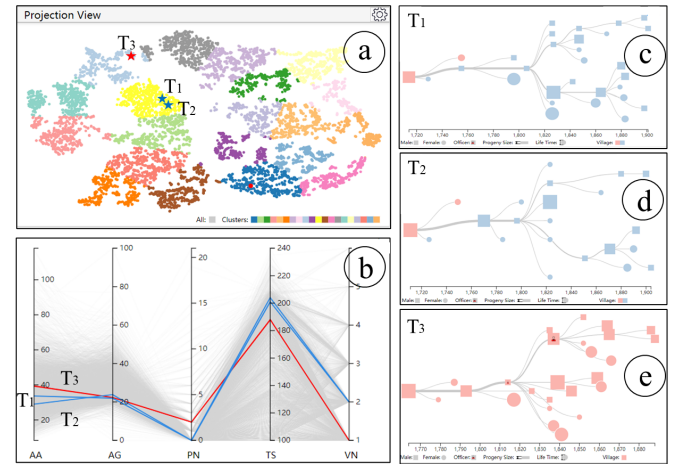


Fig. 4. Exploration on the projection view (a) with structure features and attribute features considered together. (c), (d), and (e) are family trees with similar structures, but (e) is apart from the other two due to the attribute difference, which is revealed by the parallel coordinate view (b).

found that T3 differed notably in “PN”, “TS”, and “VN”. He remarked, “This highlights the strength of the CCA-based method in jointly analyzing structure and attributes.”

Then, the 2D attribute scatter-plot view in the structure panel caught the attention of E_3 . He set the x-axis to the “depth” representing the number of generations, and the y-axis to the “nodes” representing the total number of family members, as shown in Figure 6(a). When the expert clicked a point T2 in the region with a number of nodes and a small depth (Figure 6(a)), he found this was a typical inclined family. As shown in Figure 6(c), the eldest family member of the second generation had many descendants, and he also had the experience of being an official. “This is very much in line with the Chinese tradition of primogeniture.” the expert commented. Then the expert clicked another point T1, a single-lineage family with many generations but few members, as shown in Figure 6(b). “This overview makes exploring distinctive families much easier.” E_2 praised.

E_3 tested the matching function by selecting a single-lineage family (Figure 6(b)) as the target graph and applying three similarity measures: structure, attribute, and “structure + attribute”. The matching results were shown in the candidate view (Figure 5(a)) and the parallel coordinate view (Figure 5(b)). From the candidate view, he observed that our method preserved structural similarity better than “Str” and “Attr”. In the parallel coordinate view, “Str” showed a random attribute distribution, while our method and “Attr” had similar patterns. Both performed well on “AA”, “PN”, and “VN”; “Attr” was better on “TS”, but slightly worse on “AG”. E_3 concluded, “The CCA-based method outperforms traditional structure- or attribute-based approaches in balancing structural similarity and attribute consistency.”

2) Co-author network exploration: Experts E_1 and E_2 explored the co-author network dataset D2. As members of two research groups with collaboration ties, they aimed to find similar cooperation patterns. They sketched a structure with two core nodes connecting star-shaped groups (Figure 7(b)). They adjusted the value ranges on attributes through the attribute panel, as shown in Figure 7(a). For example, the average num-

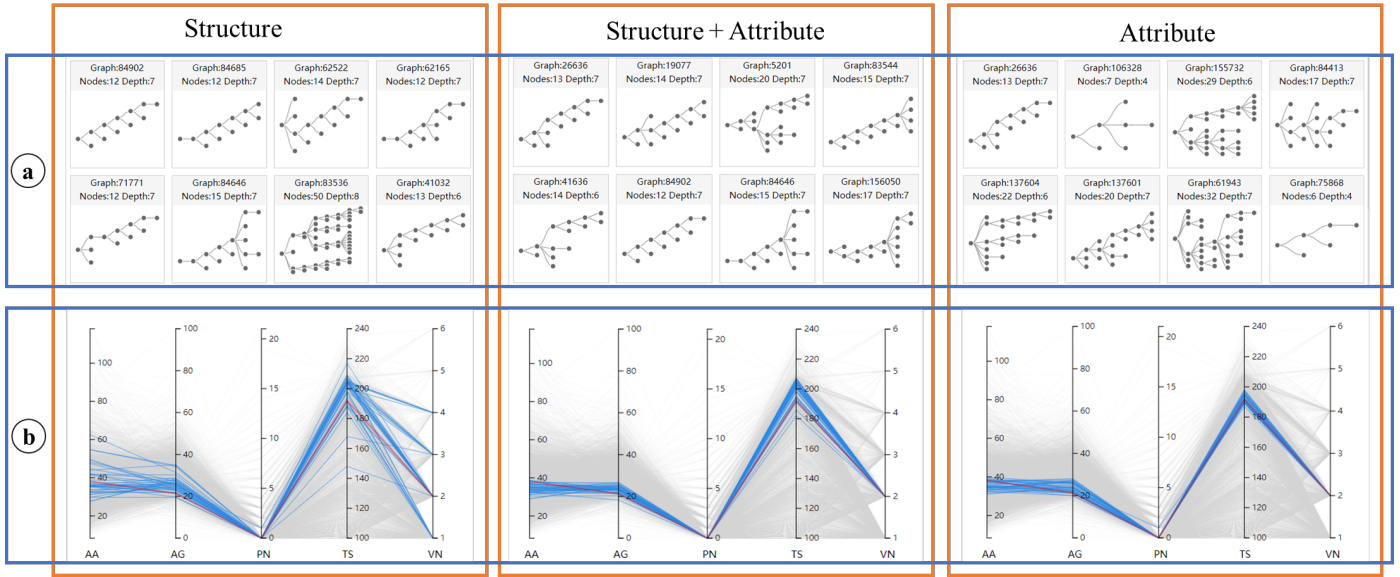


Fig. 5. Given the target graph in Figure 6(b), the matching results under different similarity measures are generated, which are evaluated in terms of structure similarity through the candidate view (a), and attribute similarity through the parallel coordinate view (b).

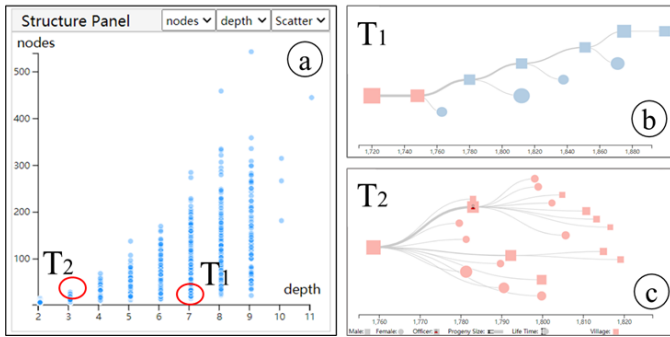


Fig. 6. Two graphs with special features are selected from the 2D attribute scatter-plot view. (a) The x-axis represents the depth (generations) of the family tree, and the y-axis represents the number of nodes in the family tree. (b) is a single-linage family with a small number of nodes and a large depth. (c) is an inclined family, with a relatively large number of nodes and a small depth.

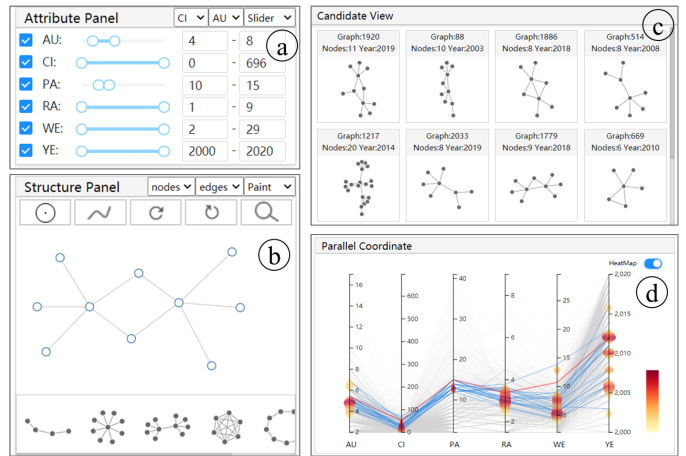


Fig. 7. Definition and matching of a target graph. (a) and (b) enable users to specify the attribute and structure of the target graph. (c) and (d) show the detailed information of matching results.

ber of authors (AU) listed in each paper was set between 4 and 8. The total number of papers published by the group (PA) was set between 10 and 15. After the structures and attributes were defined, E_1 clicked the buttons “Project” and “Match” in turn. The matching results were displayed in the candidate view, as shown in Figure 7(c). Among them, the best one was shown in the target view as shown in Figure 8(a). In the target view: node size encodes citation count; outer ring fill ratio shows paper count; node color reflects average author rank (darker = lower rank); and edge width indicates collaboration frequency.

E_2 noticed a node labeled “Huamin Qu” (red triangle) with the darkest color and largest radius in Figure 8(a). He speculated “this author may be a corresponding author with many cited papers and broad collaborations.”. E_1 then explored other graphs and found that Shixia Liu, Huamin Qu, Nan Cao, and Yingcai Wu appeared frequently. Shixia Liu was marked as a core author (red star) in multiple graphs (Figure 8(a, b, c)),

and Yingcai Wu and Huamin Qu often appeared as core authors together. E_2 noted that “senior scholars often maintain long-term collaborations.”. They also found that “Liu’s co-author networks from 2010–2013 matched, but not in 2014. By querying her name, they examined her 2014 co-author network in Figure 3(d), discovering her team expanded from two to three groups.”.

After exploring the matching results carefully, E_1 and E_2 paid attention to Yang Liu (highlighted by a red arrow) who appeared in both the cooperation networks of 2007 and 2013. They were surprised to find that though the two cooperation networks were very similar in terms of structure and attributes, other members of the two networks didn’t overlap, as shown in Figure 8(c) and Figure 8(d). Then E_2 clicked the “Table” button in the target view to check his published papers. He found that Yang Liu focused on geometric modeling and processing

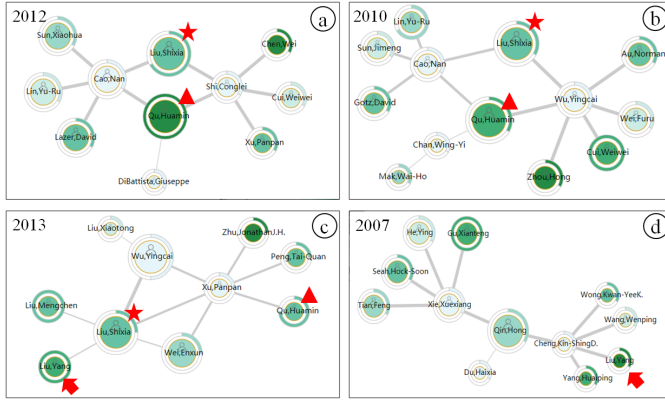


Fig. 8. Comparison of four matching results in Figure 7(c).

in 2007, and he collaborated with experts in this field. In 2013, he turned his attention to visualization and closed cooperation with experts in the field of visualization. Finally, the experts concluded that our matching model was conducive to learn the correlation between structures and attributes, and improved the accuracy of matching results in terms of both structure similarity and attribute similarity.

3) In addition to visual and quantitative validation, we further evaluated the runtime performance of our framework and baseline methods. The runtime comparison was conducted on both the Genealogy and Co-author datasets with different values of k , using a workstation with an AMD R7 CPU and 32GB of RAM. As shown in Table 3, our method achieves comparable efficiency to traditional approaches while delivering superior graph matching quality. This demonstrates that the proposed framework is not only accurate and interpretable but also computationally feasible for practical usage.

Dataset	k	Str	Attr	Our	DC	IDC
Genealogy	20	75.085	73.812	79.087	72.112	72.661
	40	75.047	75.857	79.010	72.328	73.785
	60	75.147	74.011	79.268	72.594	74.339
Co-Author	5	13.356	14.722	15.040	13.438	13.669
	10	13.449	14.894	15.041	13.561	14.493
	15	13.331	14.915	15.041	14.602	14.984

Table 3. System Time Consumption

7. Discussion

In this section, we summarize the limitations of our modular graph matching framework and discuss its generalizability and the scope of the visual analytics system.

7.1. Limitations of the Framework

The proposed graph matching framework adopts graph2vec for structural embedding and Canonical Correlation Analysis (CCA) for aligning structural and attribute representations. This design reflects a decoupled and modular approach to integrating heterogeneous graph information, in which each component

can be independently configured or substituted. For experimental validation, we selected baseline strategies such as direct and indirect concatenation to emphasize the contribution of the proposed alignment mechanism.

While effective in demonstrating the feasibility of modular structure-attribute integration, this design presents several limitations. First, graph2vec encodes graph-level structure into a fixed embedding space, which may not capture finer-grained topological semantics or hierarchical substructures as effectively as more expressive neural encoders. Second, the use of CCA provides a computationally efficient and analytically tractable means for cross-view alignment, but is limited in its capacity to model non-linear or high-order dependencies between structural and attribute spaces.

Recent developments in graph neural networks (GNNs) offer a powerful end-to-end paradigm for integrating structure and attributes through message passing. While potentially yielding better performance, GNN-based models often act as black boxes, making it difficult to interpret or intervene in the matching process. In contrast, our framework adopts a modular design that separates structure encoding (e.g., via graph2vec) and cross-view alignment (e.g., via CCA), offering greater transparency and flexibility.

This design is particularly advantageous in scenarios requiring interpretability and domain knowledge integration, such as financial risk control or medical analysis, where embedding semantics must remain accessible and adjustable. The modular nature also allows each component to be replaced with more advanced alternatives, including GNNs, non-linear alignments, or kernel-based methods, depending on data characteristics or task requirements.

Additionally, to further enhance adaptability, a user-controllable weighting mechanism could be introduced to balance structural and attribute importance during matching. Although not implemented in the current version, such extensions can be naturally supported by our flexible architecture.

7.2. Generalizability and Visual design

While our framework is designed to be modular and extensible, its application to graph datasets from other domains involves several practical considerations. In particular, structural embeddings can be reused across domains, but attribute representations often vary significantly in terms of dimensionality, semantics, and availability. To accommodate such variability, preprocessing steps such as attribute normalization, feature selection, or domain-specific encoding may be required. The alignment module (e.g., CCA) assumes a common latent space for structure and attributes, so adapting the framework to new domains may involve retraining or substituting the alignment model based on the statistical properties of the new data.

Regarding the visual analytics component, it is primarily developed to support the evaluation and interpretation of our graph matching method. The interface facilitates interaction with matched graph pairs and highlights attribute-structure correspondence, but it is not designed as a domain-agnostic tool. As such, customization for new datasets may require direct

modification of interface parameters or code-level adjustments, particularly when attribute types, graph schemas, or visual encoding conventions differ. Although the current system supports basic parameter configuration (e.g., threshold tuning, similarity metrics), deeper adaptation still relies on developer intervention rather than end-user customization.

In addition, while the visual interface qualitatively enhances interpretability and supports human-in-the-loop analysis, its effectiveness has not been formally quantified. Future work could explore user studies or task-based evaluations to assess how the visual interaction aids understanding, decision-making, or performance in different matching scenarios.

8. Conclusion

In this paper, Graph2vec is utilized to represent the structures of graphs, and a Canonical Correlation Analysis model is designed to combine the structure and attribute information into a synthetic embedding space. A distance-based graph matching scheme is employed to quickly retrieve graphs sharing similar structures and attributes. In addition, a set of visual interfaces are provided enabling users to interactively perform graph matching and visually evaluate the similarity of matching results from various perspectives. Quantitative comparisons and case studies based on real-world datasets have demonstrated the effectiveness of our system.

9. Acknowledgements

This work was supported in part by the National Natural Science Foundation of China (No.62277013, No.62177040), the National Statistical Science Research Project (No.2023LZ035), China (No.2023C01119), the Zhejiang Provincial Science and Technology Plan Project (2023C01120), the Public Welfare Plan Research Project of Zhejiang Provincial Science and Technology Department (No.LTGG23H260003, No.LGF22F020034), Open Project Program of the State Key Laboratory of CAD&CG (No.A2301), Zhejiang Provincial Natural Science Foundation of China (No.LTGG24F020006).

10. Appendix

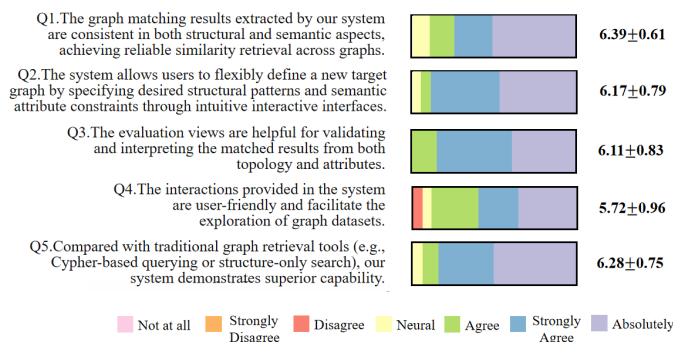


Fig. 9. Description and results of questionnaires. The right column denotes Mean ± SD.

To evaluate the usability and effectiveness of our system, we conducted a user study with 18 participants (11 men, 7 women), including graduate students and researchers in computer science, data analysis, and visualization. All had basic experience with graph concepts but no prior exposure to our system. Before the study, participants received a brief tutorial on core functions such as projection-based graph matching, target graph customization, and evaluation via coordinated views. Most became familiar with the system after 15–25 minutes of guided practice. Participants then completed exploratory tasks based on real datasets (genealogies and co-author networks), covering core scenarios from Section 3: structure-based retrieval, semantic filtering, target graph construction, and matching evaluation. Each task had a 90-second time limit, during which task status and duration were recorded. The overall task completion rate exceeded 85%. Tasks involving pure structural exploration were fastest, while those requiring attribute-structure synchronization and manual customization took longer, reflecting their complexity. Afterward, participants completed a post-study questionnaire featuring five 7-point Likert scale questions on system capabilities and usability, covering retrieval, visual design, interaction, case studies, and comparisons. As shown in Figure 9, feedback was highly positive. Q1–Q3 all scored above 6.0, confirming the effectiveness of attribute-structure synchronization and evaluation features. Q4 received slightly lower ratings, with some users noting a learning curve related to layout and projection concepts. In Q5, many appreciated the system’s seamless integration of attributes and structure—something lacking in most traditional tools.

References

- [1] D. Liu, G. Yang, Y. Wang, H. Jin, E. Chen, How to protect ourselves from overlapping community detection in social networks, *IEEE Transactions on Big Data* 8 (4) (2022) 894–904.
- [2] S. Henna, S. K. Kalliadan, Enterprise analytics using graph database and graph-based deep learning, *arXiv preprint arXiv:2108.02867* (2021).
- [3] S. Ji, S. Pan, E. Cambria, P. Martinen, S. Y. Philip, A survey on knowledge graphs: Representation, acquisition, and applications, *IEEE transactions on neural networks and learning systems* 33 (2) (2021) 494–514.
- [4] N. M. Kriege, F. D. Johansson, C. Morris, A survey on graph kernels, *Applied Network Science* 5 (1) (2020) 1–42.
- [5] J. Song, J. Song, X. Yuan, X. He, X. Zhu, Graph representation-based deep multi-view semantic similarity learning model for recommendation, *Future Internet* 14 (2) (2022) 32.
- [6] Z. Lan, B. Hong, Y. Ma, F. Ma, More interpretable graph similarity computation via maximum common subgraph inference, *IEEE Transactions on Knowledge and Data Engineering* 36 (11) (2024) 6588–6599.
- [7] S. Saadah, M. F. E. Saputro, K. R. S. Wiharja, Graph edit distance optimized with greedy algorithm for similarity search in business process graphs, in: *2020 International Conference on Data Science and Its Applications (ICoDSA)*, 2020, pp. 1–7.
- [8] S. Mahmood, K. Mueller, Interactive subspace cluster analysis guided by semantic attribute associations, *IEEE Transactions on Visualization and Computer Graphics* (2023).
- [9] P. Zhou, Y. Zhang, F. Chen, K. Pang, M. Lu, Heterogeneous ppi network representation learning for protein complex identification, in: *International Symposium on Bioinformatics Research and Applications*, Springer, 2022, pp. 217–228.
- [10] Z. Deng, D. Weng, J. Chen, R. Liu, Z. Wang, J. Bao, Y. Zheng, Y. Wu, Airvis: Visual analytics of air pollution propagation, *IEEE transactions on visualization and computer graphics* 26 (1) (2019) 800–810.
- [11] C. Wangmo, L. Wiese, Subtempora: A hybrid approach for optimising

- subgraph searching, in: International Conference on Data Management Technologies and Applications, Springer, 2021, pp. 66–89.
- [12] F. Bi, L. Chang, X. Lin, L. Qin, W. Zhang, Efficient subgraph matching by postponing cartesian products, in: Proceedings of the 2016 International Conference on Management of Data, 2016, pp. 1199–1214.
 - [13] C. Q. Cheng, K. S. Wong, L. K. Soon, l2match: Optimization techniques on subgraph matching algorithm using label pair, neighboring label index, and jump-redo method, in: 2024 International Conference on Electronics, Information, and Communication (ICEIC), IEEE, 2024, pp. 1–6.
 - [14] N. Karpov, Q. Zhang, Syncsignature: a simple, efficient, parallelizable framework for tree similarity joins, Proceedings of the VLDB Endowment 16 (2) (2022) 330–342.
 - [15] Y. Chai, J. Li, Q. Zhang, J. Ge, X. Wang, Asm: Adaptive subgraph matching via efficient compression and label filter, in: Asia-Pacific Web (AP-Web) and Web-Age Information Management (WAIM) Joint International Conference on Web and Big Data, Springer, 2024, pp. 30–42.
 - [16] Y. Wang, C. Jin, S. Cai, Pathlad+: Towards effective exact methods for subgraph isomorphism problem, Artificial Intelligence 337 (2024) 104219.
 - [17] C. Q. Cheng, K. S. Wong, L. K. Soon, l2match: Optimization techniques on subgraph matching algorithm using label pair, neighboring label index, and jump-redo method, arXiv preprint arXiv:2311.16603 (2023).
 - [18] W. Yang, C. Chen, J. Zhu, L. Li, P. Liu, S. Liu, A survey of visual analytics research for improving training data quality, Journal of Computer-Aided Design & Computer Graphics 35 (11) (2023) 1629–1642.
 - [19] F. Lyu, C. Chen, J. Zhang, X. Feng, Z. Tang, Visualization for supercomputer system: A survey, Journal of Computer-Aided Design & Computer Graphics 36 (3) (2024) 321–335.
 - [20] R. Pienta, A. Tamersoy, A. Endert, S. Navathe, H. Tong, D. H. Chau, Visage: Interactive visual graph querying, in: Proceedings of the International Working Conference on Advanced Visual Interfaces, 2016.
 - [21] N. Cao, Y.-R. Lin, L. Li, H. Tong, g-miner: Interactive visual group mining on multivariate graphs, in: Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems, 2015.
 - [22] P. C. Wong, D. Haglin, D. Gillen, D. Chavarria, V. Castellana, C. Joslyn, A. Chappell, S. Zhang, A visual analytics paradigm enabling trillion-edge graph exploration, in: 2015 IEEE 5th Symposium on Large Data Analysis and Visualization (LDAV), IEEE, 2015.
 - [23] W. Chen, F. Guo, D. Han, J. Pan, X. Nie, J. Xia, X. Zhang, Structure-based suggestive exploration: a new approach for effective exploration of large networks, IEEE transactions on visualization and computer graphics 25 (1) (2018) 555–565.
 - [24] R. Pienta, F. Hohman, A. Endert, A. Tamersoy, K. Roundy, C. Gates, S. Navathe, D. H. Chau, Vigor: interactive visual exploration of graph query results, IEEE transactions on visualization and computer graphics 24 (1) (2017) 215–225.
 - [25] H. Song, Z. Dai, P. Xu, L. Ren, Interactive visual pattern search on graph data via graph representation learning, IEEE Transactions on Visualization and Computer Graphics 28 (1) (2021) 335–345.
 - [26] Z. Zhou, C. Shi, X. Shen, L. Cai, H. Wang, Y. Liu, Y. Zhao, W. Chen, Context-aware sampling of large networks via graph representation learning, IEEE Transactions on Visualization and Computer Graphics 27 (2) (2020) 1709–1719.
 - [27] Z. Zhou, H. Wang, Z. Zhao, F. Zheng, Y. Wang, W. Chen, Y. Wang, Chartkg: A knowledge-graph-based representation for chart images, IEEE Transactions on Visualization and Computer Graphics (2024).
 - [28] Z. Zhou, L. Sun, H. Wang, W. Yu, Y. Liu, X. Zhang, Y. Wang, W. Chen, imgc: Interactive multiple graph clustering with constrained laplacian rank, IEEE Transactions on Human-Machine Systems 53 (2) (2022) 427–437.
 - [29] M. Grohe, word2vec, node2vec, graph2vec, x2vec: Towards a theory of vector embeddings of structured data, in: Proceedings of the 39th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems, 2020, pp. 1–16.
 - [30] M. Tang, C. Yang, P. Li, Graph auto-encoder via neighborhood wasserstein reconstruction, arXiv preprint arXiv:2202.09025 (2022).
 - [31] R. Yang, J. Shi, X. Xiao, Y. Yang, S. S. Bhowmick, J. Liu, Pane: scalable and effective attributed network embedding, The VLDB Journal 32 (6) (2023) 1237–1262.
 - [32] J. Wang, J. Guo, Y. Sun, J. Gao, S. Wang, Y. Yang, B. Yin, Dggn: Decoupled graph neural networks with structural consistency between attribute and graph embedding representations, IEEE Transactions on Big Data (2024).
 - [33] C. Wang, J. Han, D4scivis: A state-of-the-art survey on deep learning for scientific visualization, IEEE Transactions on Visualization and Computer Graphics (2022).
 - [34] H. Li, H. Wang, Z. Yang, M. Odagaki, Francis, nadime and green, alastair and guagliardo, paolo and libkin, leonid and lindaaker, tobias and marsault, victor and plantikow, stefan and rydberg, mats and selmer, petra and taylor, andrés, in: Proceedings of the 2018 international conference on management of data, 2018.
 - [35] H. Thakkar, D. Punjani, S. Auer, M.-E. Vidal, Towards an integrated graph algebra for graph pattern matching with gremlin, in: D. Benslimane, E. Damiani, W. I. Grosky, A. Hameurlain, A. Sheth, R. R. Wagner (Eds.), Database and Expert Systems Applications, Springer International Publishing, Cham, 2017, pp. 81–91.
 - [36] A. Hogan, A. Hogan, Sparql query language, The Web of Data (2020) 323–448.
 - [37] A. Khan, Y. Wu, C. C. Aggarwal, X. Yan, Nema: Fast graph search with label similarity, Proceedings of the VLDB Endowment 6 (3) (2013) 181–192.
 - [38] P. Zhao, J. Han, On graph query optimization in large networks, Proceedings of the VLDB Endowment 3 (1-2) (2010) 340–351.
 - [39] K. D. Doan, S. Manchanda, S. Mahapatra, C. K. Reddy, Interpretable graph similarity computation via differentiable optimal alignment of node embeddings, in: Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval, 2021, pp. 665–674.
 - [40] O.-H. Kwon, T. Crnovrsanin, K.-L. Ma, What would a graph look like in this layout? a machine learning approach to large graph visualization, IEEE transactions on visualization and computer graphics 24 (1) (2017) 478–488.
 - [41] H. Li, Y. Wang, A. Wu, H. Wei, H. Qu, Structure-aware visualization retrieval, in: Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems, 2022.
 - [42] M. E. Sargin, Y. Yemez, E. Erzin, A. M. Tekalp, Audiovisual synchronization and fusion using canonical correlation analysis, IEEE transactions on Multimedia 9 (7) (2007) 1396–1403.
 - [43] N. Shervashidze, P. Schweitzer, E. J. Van Leeuwen, K. Mehlhorn, K. M. Borgwardt, Weisfeiler-lehman graph kernels., Journal of Machine Learning Research 12 (9) (2011).
 - [44] M. Ester, H.-P. Kriegel, J. Sander, X. Xu, et al., A density-based algorithm for discovering clusters in large spatial databases with noise, in: kdd, Vol. 96, 1996.
 - [45] J. MacQueen, et al., Some methods for classification and analysis of multivariate observations, in: Proceedings of the fifth Berkeley symposium on mathematical statistics and probability, Vol. 1, Oakland, CA, USA, 1967.
 - [46] T. T. Cai, R. Ma, Theoretical foundations of t-sne for visualizing high-dimensional clustered data, Journal of Machine Learning Research 23 (301) (2022) 1–54.
 - [47] J. Lee, C. D. Campbell, S. Chen, China multi-generational panel dataset, liaoning (cmgpd-ln), 1749-1909, Data Sharing for Demographic Research (DSDR) (2010).
 - [48] M. Li, S. Lu, L. Zhang, Y. Zhang, B. Zhang, A community detection method for social network based on community embedding, IEEE Transactions on Computational Social Systems 8 (2) (2021) 308–318.
 - [49] A. Moscatelli, J. Piquenot, M. Bérar, P. Héroux, S. Adam, Graph node matching for edit distance, Pattern Recognition Letters 184 (2024) 14–20.
 - [50] P. Wills, F. G. Meyer, Metrics for graph comparison: a practitioner's guide, Plos one 15 (2) (2020) e0228728.